LONG-TERM MEMORY ENGINE FOR AI AGENTS

A PROJECT REPORT

Submitted by

M. KRITHIKRAJ	**2023115004**
ALSTON S	**2023115006**
ARYAN I	**2023115021**
HARI SRINIVASAN	**2023115022**

submitted to the Faculty of

COMPUTER SCIENCE AND INFORMATION ENGINEERING

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI – 600 025

OCTOBER 2026

ANNA UNIVERSITY

CHENNAI – 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **LONG-TERM MEMORY ENGINE FOR AI AGENTS** is the bona fide work of **M. KRITHIKRAJ (2023115004), ALSTON S (2023115006), ARYAN I (2023115021)** and **HARI SRINIVASAN (2023115022)** who carried out the project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: Chennai

DATE:

DR. BAMA SRINIVASAN
PROJECT GUIDE
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

COUNTERSIGNED

HEAD OF THE DEPARTMENT
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

ABSTRACT

AI agents that don't have persistent memory forget instructions and project decisions once a session ends, and when an instruction changes they often keep using the old one. For this project we built a long-term memory engine in Rust for such agents. What it remembers carries over between sessions, can be corrected later, and can be traced back to the original messages.

The engine has five modules: source acquisition and processing, knowledge extraction and consolidation, temporal knowledge and graph management, hybrid retrieval and grounded reasoning, and model training and evaluation. Original messages are stored unchanged as source events. Assertions (typed statements about the user or project) are extracted along with the exact source quote they came from and stored as versions, so a correction replaces an old value without deleting it. Retrieval combines keyword, vector, time and graph search using Reciprocal Rank Fusion and returns each result with its source quotes. The extraction and answering is done by a small local Qwen3-4B model that we fine-tuned with QLoRA.

We tested the engine on LoCoMo, a public benchmark of ten long conversations (1,540 questions), using the same reader model for every setup and two model judges. With the engine the reader answered 83.1% of the questions correctly, compared with 80.0% for plain retrieval over the raw text, 43.0% when keeping only the last 8,192 tokens, and 0.3% with no memory. Giving the reader the whole conversation scored 89.0%, but used more than three times as many tokens. All 99,971 source quotes returned by the engine matched the conversation exactly. Our own scripted tests showed that corrections keep their history and that the engine refuses to answer without evidence, and they also caught a date bug, which we fixed after the test.

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. Bama Srinivasan, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, for her guidance, regular reviews and constructive suggestions throughout this project. We also thank the Department of Information Science and Technology for the facilities and support provided.

We acknowledge the open-source and research community whose published work and software made this project possible, including the authors of the systems and papers cited in this report.

M. KRITHIKRAJ
ALSTON S
ARYAN I
HARI SRINIVASAN

TABLE OF CONTENTS

ABSTRACT iii
LIST OF TABLES vii
LIST OF FIGURES viii
LIST OF ABBREVIATIONS ix
GLOSSARY x

1 INTRODUCTION **1**
1.1 BACKGROUND 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 2
1.4 CHALLENGES FACED 3
1.5 ORGANISATION OF THE REPORT 3

2 LITERATURE SURVEY **4**
2.1 LONG CONTEXT AND EXTERNAL MEMORY 4
2.2 MEMORY SYSTEMS 4
2.3 METHODS USED IN THIS PROJECT 6
2.4 COMPARISON AND GAP 6

3 SYSTEM DESIGN **7**
3.1 DESIGN REQUIREMENTS 7
3.2 OVERALL ARCHITECTURE 7
3.3 M1 SOURCE ACQUISITION AND PROCESSING 7
3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION 8
3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT 9
3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING 10
3.7 M5 MODEL TRAINING AND EVALUATION 10

4 IMPLEMENTATION **12**
4.1 IMPLEMENTATION ENVIRONMENT 12
4.2 ALGORITHMS 12
4.2.1 Priority-Based Context Selection 12
4.2.2 Entity Resolution and Memory Versioning 13
4.2.3 Memory Consolidation and Invalidation 13

4.2.4 Hybrid Retrieval and Graph Traversal 14
4.2.5 Reciprocal Rank Fusion and Context Packing 14
4.2.6 Source-Grounded Answer Generation 14
4.3 INTEGRATION WITH THE PI AGENT 15

5 RESULTS, PERFORMANCE ANALYSIS AND COMPARISON 16
5.1 DATASETS 16
5.2 FINE-TUNING AND MODEL SELECTION 17
5.3 LIVE DEMONSTRATION 18
5.4 BENCHMARK SETUP 19
5.5 DEVELOPMENT ITERATIONS 21
5.6 RESULTS 21
5.7 PERFORMANCE ANALYSIS 23
5.8 PROPERTY TESTS 24
5.9 COMPARISON WITH EXISTING SYSTEMS 24
5.10 LIMITATIONS AND FUTURE WORK 26

REFERENCES 27

LIST OF TABLES

2.1 Comparison of existing memory systems 6

5.1 LoCoMo results by condition (all ten conversations, categories 1–4) 22
5.2 Accuracy by LoCoMo question category (Jev, %) 22
5.3 Published LoCoMo results [1] beside this work (GPT-4o-mini judge) 25

LIST OF FIGURES

3.1 Overall system architecture 8
3.2 Module 1: Source Acquisition and Processing 9
3.3 Module 2: Knowledge Extraction and Consolidation 9
3.4 Module 3: Temporal Knowledge and Graph Management 10
3.5 Module 4: Hybrid Retrieval and Grounded Reasoning 11
3.6 Module 5: Model Training and Evaluation 11

5.1 General-purpose training rows across the 16 domains 17
5.2 Demo: recording details, then recalling them in a new session 18
5.3 Demo: recalled answer and the corrected assertion in the graph 19

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
DMR	Deep Memory Retrieval
GGUF	GPT-Generated Unified Format
HTTP	Hypertext Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MVCC	Multi-Version Concurrency Control
NELL	Never-Ending Language Learner
NF4	4-bit NormalFloat
p95	95th percentile
PROV	W3C Provenance data model
QLoRA	Quantised Low-Rank Adaptation
RRF	Reciprocal Rank Fusion
SHA-256	Secure Hash Algorithm, 256-bit
SLM	Small Language Model
SQL	Structured Query Language
W3C	World Wide Web Consortium

GLOSSARY

Some everyday computing words are used with a narrower meaning in this report. The meanings below are the ones we use throughout.

Memory	Knowledge about a user or project that the agent keeps across sessions (assertions, inferences and the source events they came from). It does *not* mean RAM, cache, disk or any other computer memory.
Long-term memory	Memory that survives after a session ends, as opposed to what is only in the current prompt.
Agent	An AI program (here, the Pi coding agent) that talks to the user and carries out tasks using a language model and tools.
Harness	The program around the model that runs the agent: it builds prompts, calls tools and in our case talks to the memory service.
Session	One continuous conversation between the user and the agent. A new session starts with an empty prompt.
Context / context window	The text the model can read at one time (its prompt). Its size is limited and measured in tokens.
Token	A small piece of text (a word or part of a word) that the model reads. Budgets and limits are counted in tokens.
Namespace	A label such as `user:maya` that keeps one user's or project's memory separate from another's.
Source event	A single message or tool result (with role and timestamp) exactly as it was sent, stored before anything is extracted from it. "Event" is short for source event.

Episode	One batch of source events saved together by a single retain request.
Source chunk	A piece of a source event's text that is stored and embedded for search.
Source quote	The exact passage of a source event that an assertion cites. It must appear word for word in the original text.
Spool	A local folder of files on the client where batches wait if the memory service can't be reached.
Job / worker	A job is a stored task (e.g. "extract assertions from this episode"); a worker is a background process that picks up and runs jobs.
Lease	A time-limited lock a worker takes on a namespace so that two workers don't process it at the same time.
Claim	A candidate assertion proposed by the model before the system has checked and stored it.
Assertion	A typed, temporal statement about an entity that the model extracted from source events, e.g. "Maya lives in Bengaluru", stored with its source quote and validity interval. It can be wrong, contested, superseded or retracted.
Attribute	The assertion kind for a stable property of an entity, such as a home city or an employer (stored as kind `attribute`).
Inference	An assertion (kind `inference`) derived from at least two supporting assertions, e.g. "Maya works remotely as a software engineer at NovaTech".
Entity	A person, project, place or thing that assertions are about.
Consolidation	The step that combines related assertions into an inference.

Invalidation	Marking an inference stale when one of the assertions it depends on changes.
Supersede	To replace an assertion's value with a newer one while keeping the old assertion as history.
Validity interval	The time range during which an assertion was true, e.g. Chennai from 1 to 10 October.
Embedding / vector	A list of numbers representing the meaning of a text, used to find similar texts.
Retrieval / recall	Finding the stored memories relevant to a question and returning them to the agent.
Projection	The copy of assertions and entities kept in Neo4j as a graph. It can be rebuilt from PostgreSQL.
Reflection	An optional step where the model writes an answer from retrieved memories, with citations.
Grounded answer	An answer in which every statement is backed by a cited, retrieved memory.
Evidence	Only in its ordinary sense: whatever is cited in support of a conclusion, such as the passages a reflected answer cites. A stored message is a source, not evidence.
Fine-tuning	Further training of an existing model on our own data so it does a specific task better.
Adapter	The small set of extra weights trained by LoRA/QLoRA; the base model itself stays frozen.
Quantisation	Storing model weights with fewer bits (e.g. 4-bit or 8-bit) so the model uses less memory and runs faster. Here "memory" does mean computer memory.
Teacher model	A larger model (GPT-5.6-sol) used to write the expected outputs for our training data.

Why “source” and “assertion”, not “evidence” and “fact”. Evidence is a role, not a kind of record: a message becomes evidence only when something cites it in support of a conclusion. A fact is something established, whereas our extracted units are produced by a model and can be wrong, contested, superseded or retracted. We therefore follow the knowledge-representation usage of claims and their sources. Wikidata stores each statement (claim) with references to the sources that back it [16], nanopublications keep an assertion separate from its provenance, the W3C PROV ontology links a derived entity to what it came from with `prov:wasDerivedFrom` and to a quoted passage with `prov:wasQuotedFrom` [17], and NELL holds extracted statements as “candidate beliefs” until enough support promotes them [18]. We also avoid “observation” for derived units, because Generative Agents [19] uses it for raw perceptions and “reflection” for derived ones (“reflection” already names question answering in our system). Zep/Graphiti [5], Mem0 [1] and Hindsight [7] call their extracted units “facts”; we deliberately depart from that usage.

CHAPTER 1

INTRODUCTION

AI agents are now used for coding projects that go on for weeks over many sessions, and every new session starts with an empty prompt. Whatever the user told the agent last week is simply gone unless the user types it in again. This project gives the agent a long-term memory outside the model.

1.1 BACKGROUND

The obvious fix is to paste the whole history into the prompt every time. This does not work well. Liu et al. [2] showed that language models use long inputs unevenly and miss details in the middle of the context, and old statements sit right next to their corrections with no rule for which one is current.

So a memory component has to decide what to keep and what to give back for a question. We built this as a separate service for a coding agent (Pi [9]). It stores the original messages, extracts assertions from them, remembers when an assertion changed, and returns only the relevant assertions along with the quote they came from. In this report *memory* always means this kind of stored knowledge, not RAM; other terms such as *source* and *assertion* are explained in the Glossary.

The engine has five modules: M1 Source Acquisition and Processing saves the original messages, M2 Knowledge Extraction and Consolidation extracts assertions from them, M3 Temporal Knowledge and Graph Management stores the assertions with their history, M4 Hybrid Retrieval and Grounded Reasoning finds the right assertions for a question, and M5

Model Training and Evaluation trains the small local model the others use. M1 to M4 make up the memory service and M5 is an offline pipeline.

1.2 PROBLEM STATEMENT

Agents without persistent memory lose instructions and project decisions when a task continues across sessions, and when an instruction changes they may reuse the old value. Users end up repeating context and fixing the same mistakes again.

Throughout the report we use one small example, a user called Maya. On 1 October 2026 she tells the agent "My name is Maya. I live in Chennai. I work as a software engineer at NovaTech, and I work remotely." (message E1). On 10 October she says she has moved to Bengaluru (E3). On 13 October she asks what the agent remembers about where she lives, her previous city and her work (E5). An agent with no memory can't answer E5, and one that only kept E1 would confidently say Chennai, which is worse. We want it to know Bengaluru replaced Chennai on 10 October and still keep Chennai as her previous city.

1.3 OBJECTIVES

1. Retain useful information for AI agents across sessions.
2. Keep the updated instruction while preserving the earlier versions.
3. Link every stored assertion back to the original message so it can be verified.
4. Fine-tune a small local model to extract assertions, combine them into inferences and answer with citations.
5. Retrieve relevant information quickly within a limited context size.

6. Evaluate the engine with repeatable tests on a public long-conversation benchmark, LoCoMo [15].

1.4 CHALLENGES FACED

Our early models did well on generated test data but struggled on real conversations (only 18 of 33 conversation segments passed the extraction checks) and mixed up quotes, sources and dates, so we revised the data, prompts and validation several times. The model also sometimes re-extracted an outdated value, repeated messages made duplicates, and one failed job could block the rest. The hardest bug to notice was that the deployed model silently cut off inputs longer than its limit, so now we check input length and split long conversations into smaller pieces.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews existing memory systems, Chapter 3 the architecture and modules, Chapter 4 the tools and algorithms, and Chapter 5 the data, fine-tuning, demo, benchmark results and the comparison with published memory systems.

CHAPTER 2

LITERATURE SURVEY

This chapter looks at existing work on long-term memory for agents. Every number quoted here is the one reported by the authors on their own setup; we did not rerun any of these systems.

2.1 LONG CONTEXT AND EXTERNAL MEMORY

The simplest memory is to keep the entire conversation in the prompt. Liu et al. [2] found that models use information in the middle of a long context less reliably than information near the start or end, so a bigger context window does not remove the need to choose what the model reads. It is also slow (17 s p95 latency for the full history in the Mem0 paper).

All the systems below do the same basic thing: they keep memory outside the prompt and only show the model a selected part of it. Where they differ is what they store, who decides how a stored item changes (the model or the code), and whether the old value of a changed item is kept.

2.2 MEMORY SYSTEMS

MemGPT. Packer et al. [4] split the prompt into a main context and external recall and archival storage, and let the model move data between them using function calls. On deep memory retrieval they report 92.5% for GPT-4 with MemGPT against 32.1% for GPT-4 reading a summary. The catch is that a missed function call means a missed update.

Zep and Graphiti. Rasmussen et al. [5] store a temporal knowledge graph of episodes, entities and "facts" (their term). Each one has a valid time and an invalid time, and a contradicting one closes the old one instead of deleting it. They report 71.2% on LongMemEval with gpt-4o against 60.2% for the full 115k-token history, from a single run. This is closest to ours, except that in Zep the graph is the main store while we keep the real data in PostgreSQL.

Mem0. In Mem0 [1] an LLM extracts candidate "facts", compares each one with the nearest stored memories and picks one of add, update, delete or no-op. On LoCoMo they report a judge score of 66.9 against 72.9 for the full history, with p95 latency of 1.4 s against 17 s. Only one benchmark and an unnamed judge model were used, and the open-source release has since dropped update, delete and graph memory. We use the same extract-then-compare idea, but in our system the update decision is made by fixed code rules and not by the LLM for each extracted item.

Supermemory. Supermemory [6] stores atomic memories linked to source chunks, with "updates", "extends" and "derives" links and two timestamps on each. Its results are only from the vendor's own report and the engine is closed source.

Hindsight. Hindsight [7] is built around three operations, retain, recall and reflect. Recall combines four retrieval strategies using rank fusion and packs the result into a token budget. Our recall step is based on this, though the paper is not peer reviewed.

Benchmarks. LongMemEval [3] tests chat assistants on long-term interactive memory, and Zep, Hindsight and Supermemory all report on it. LoCoMo [15] evaluates very long-term conversational memory and is the only benchmark used in the Mem0 paper. The published results aren't really

comparable, since each uses its own models and judge.

2.3 METHODS USED IN THIS PROJECT

We used a few well known methods as building blocks. PostgreSQL full-text search [10] with GIN indexes is our keyword search. pgvector [11] adds vector columns and nearest-neighbour search to the same database, which we use with cosine distance on 1,024-dimensional vectors. Reciprocal Rank Fusion [12] merges several ranked lists by adding up $1/(k+rank)$ for each item; we use $k = 60$. The vectors come from a Qwen3 Embedding model [13] served through Ollama, and the memory model is fine-tuned with QLoRA [14], which trains small adapter weights on top of a frozen, quantised base model.

2.4 COMPARISON AND GAP

Table 2.1: Comparison of existing memory systems

System	Who decides an update	Main limitation
MemGPT	The model, through function calls	Edits depend on correct calls; no cost figures
Zep / Graphiti	Graph rule (validity intervals)	Single run; graph database is the main store
Mem0	The LLM, per item	One benchmark; released code differs from paper
Supermemory	Not fully documented	Closed source, vendor report only
Hindsight	Retain step	Not peer reviewed; budget not stated

Table 2.1 puts the five systems side by side. In all of them either the model decides updates or the rule is tied to a graph database. So we took the extract-then-compare idea, the validity intervals and the fused recall, but made the update rule plain service code, kept PostgreSQL as the source of truth, tied every assertion to an exact quote, and trained a small local extraction model.

CHAPTER 3

SYSTEM DESIGN

This chapter describes the design requirements, the architecture and the five modules, following the Maya example through them.

3.1 DESIGN REQUIREMENTS

We worked from a few rules. Original messages are saved exactly as sent *before* any extraction, which runs later as a queued job, so a failed model call can't lose a conversation. A changed assertion becomes a new version and the old value is kept with its validity interval. Every assertion links to an exact source quote. All data belongs to a namespace (e.g. `user:maya`), whatever we hand to a model has to fit a token budget, and models run locally on Ollama. Neo4j is only a copy that can be rebuilt from PostgreSQL.

3.2 OVERALL ARCHITECTURE

Figure 3.1 shows the architecture. The client is the Pi coding agent, whose memory client saves every message through the memory service and asks for recall before answering. The service is a Rust application with API handlers and background workers. PostgreSQL with pgvector is the main store for M1 to M4: source events and chunks, entities, assertions, quotes, vectors and the job queue. Ollama runs our fine-tuned worker model and a Qwen3 embedding model. Neo4j keeps a graph copy of assertions and entities, and M4 rechecks every graph hit against PostgreSQL. M5 runs offline, and live chats are *not* used as training data.

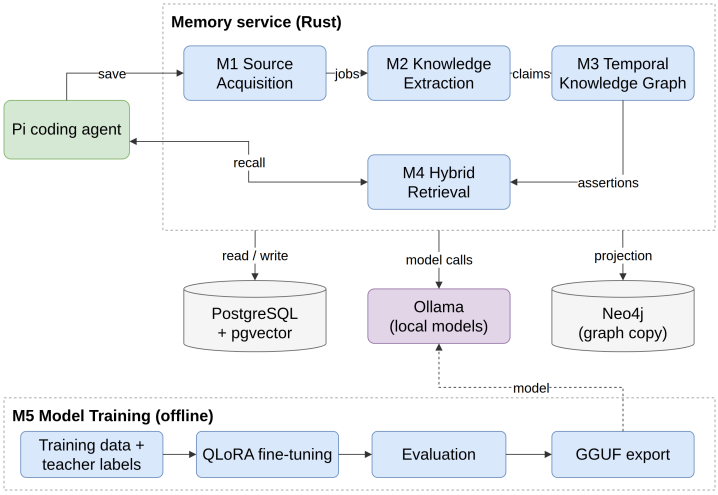


Figure 3.1: Overall system architecture

3.3 M1 SOURCE ACQUISITION AND PROCESSING

M1 saves the source events before anything else (Figure 3.2). For Maya, the 10 October conversation arrives as a retain request with namespace `user:maya`, session S2 and the events E3 (her message) and E4 (the agent's reply). The batch is first written to a local spool so nothing is lost if the service is down, and then sent to `POST /api/v2/retain`. If the same key arrives again with the same hash the existing IDs are returned, so retries don't create duplicates. The episode, events, chunks and a pending extraction job are saved in one transaction, and each chunk is embedded into a 1,024-dimensional vector. Background workers lease a namespace for 15 minutes and retry a job up to 5 times.

3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION

M2 has two stages, both using our local model through Ollama (Figure 3.3). **Extraction:** the prompt builder fits the new events and the relevant existing assertions into the model's input (Section 4.2.1). For Maya, "Maya / Chennai" is shown next to E3, and the model returns home city Bengaluru,

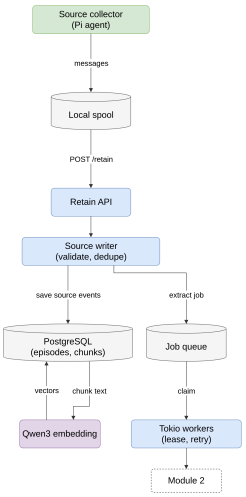


Figure 3.2: Module 1: Source Acquisition and Processing

flagged as a correction, with an exact quote. Output that fails the schema or quotes text that isn't in the source is retried up to two times. Secrets are filtered out before the claim goes to M3. **Consolidation:** after M3 commits, the model may combine two or more active assertions about the same subject into one inference. Maya's three work assertions become "Maya works remotely as a software engineer at NovaTech."

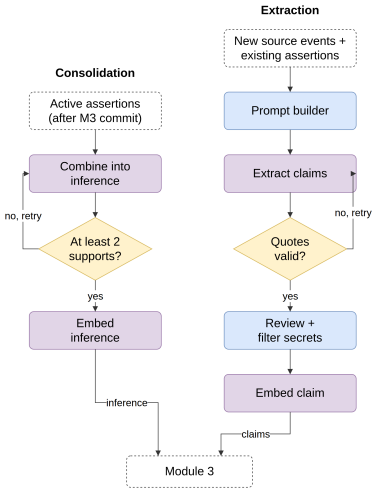


Figure 3.3: Module 2: Knowledge Extraction and Consolidation

3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT

M3 is the only module that writes assertions, inside one PostgreSQL transaction that holds a namespace lock (Figure 3.4). It first resolves the entity (Maya). If the same value already exists it is reinforced, and if the slot is empty a new assertion is made. If a single-valued slot gets a different value and the claim is a reviewed correction with confidence at least 0.8, the old assertion is superseded; otherwise both are marked contested. So R1 Chennai becomes valid from 1 to 10 October and superseded, and R2 Bengaluru is active from 10 October with the E3 quote. A projection worker then copies the change to Neo4j [8], checking revisions so an old one can't overwrite a newer one.

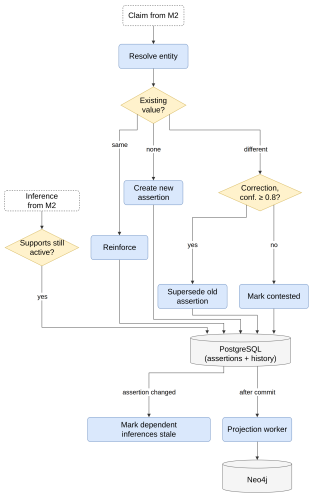


Figure 3.4: Module 3: Temporal Knowledge and Graph Management

3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING

M4 answers recall requests (Figure 3.5). On 13 October Pi sends E5 for `user:maya` with `top_k` 10, a maximum distance of 0.45 and a 2,048 token budget. The query goes to keyword, vector and (with a date window) time-based search, the top results seed a Neo4j expansion, and the lists are merged with Reciprocal Rank Fusion and packed into the budget with source quotes. An optional reflect endpoint writes a cited answer.

3.7 M5 MODEL TRAINING AND EVALUATION

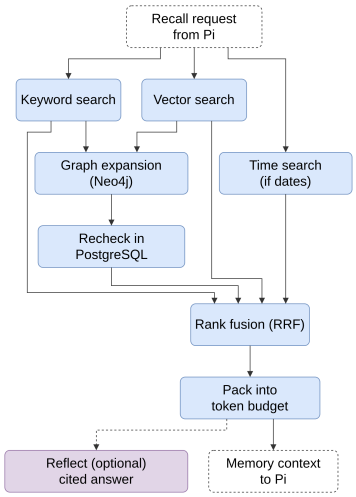


Figure 3.5: Module 4: Hybrid Retrieval and Grounded Reasoning

M5 runs offline and produces the worker model used by M2 and M4 (Figure 3.6). A generator makes short episodes across 16 domains, a teacher model labels them, and the checked labels are used to train the model with QLoRA, which is then evaluated and imported into Ollama (details in Chapter 5). In the end, when Maya asks E5, Pi answers "You now live in Bengaluru, following your move on 10 October 2026. Before that, you lived in Chennai. You work remotely as a software engineer at NovaTech."

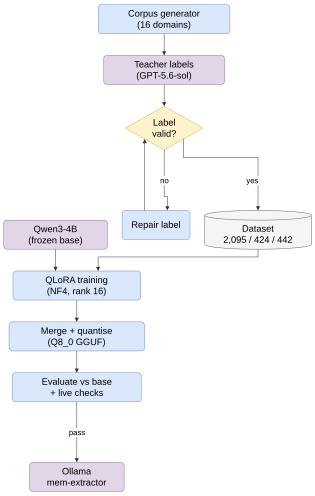


Figure 3.6: Module 5: Model Training and Evaluation

CHAPTER 4

IMPLEMENTATION

This chapter covers the tools we used, the six main algorithms (with the Maya example walked through each) and how the engine plugs into Pi.

4.1 IMPLEMENTATION ENVIRONMENT

The memory service is written in Rust, using Tokio for the background workers, Axum for the API and SQLx for PostgreSQL. PostgreSQL 17 with pgvector is the main store and does both full-text and vector search, and Neo4j 5.26 holds the graph copy. The models run locally on Ollama: our fine-tuned Qwen3-4B-Instruct-2507 extracts assertions, combines them into inferences and writes cited answers, and Qwen3-Embedding-0.6B makes the vectors. The dashboard is React + TypeScript. Training used Python, PyTorch, Transformers, PEFT and bitsandbytes, with llama.cpp to make the GGUF, and Docker Compose runs the services.

4.2 ALGORITHMS

Context selection is in M2, entity resolution and versioning in M3, consolidation and invalidation is split between M2 and M3, and retrieval, rank fusion and answer generation are in M4.

4.2.1 Priority-Based Context Selection

The extractor needs to see the assertions already stored about the same people, or it can't tell a repeat from a correction, but they may not all fit its input limit (Algorithm 4.1). When E3 arrives Maya has four stored assertions; if only three fit, the home city one ranks first because E3 mentions both Maya and Chennai, and the rest follow by recency.

Algorithm 4.1 Priority-based context selection

```
IF all existing assertions fit: RETURN all of them
Score assertions by subject and value mentions in new messages
Rank stronger matches first; break ties by recency
Binary search for the largest ranked prefix that fits
RETURN the selected assertions in their original order
```

4.2.2 Entity Resolution and Memory Versioning

The key rule is that a conflicting value never silently overwrites the old one, and the old assertion is superseded, not deleted (Algorithm 4.2). The E3 claim (home city Bengaluru, correction, confidence 0.95) conflicts with R1 Chennai and $0.95 \geq 0.8$, so R1 is superseded with validity 1–10 October and R2 Bengaluru becomes active.

Algorithm 4.2 Entity resolution and memory versioning

```
Match entity by type and normalised name or alias; else create it
IF the same assertion exists: attach the new source quote
ELSE IF no conflicting single-value assertion: store as active
ELSE IF explicit correction AND confidence >= 0.8:
    Supersede the old assertion; activate the new one
ELSE: mark both values as disputed
```

4.2.3 Memory Consolidation and Invalidation

The model proposes an inference but the code decides whether to save it (Algorithm 4.3). For Maya, A1 (NovaTech), A2 (software engineer) and A3 (remote) give inference I1. If A3 were retracted later, I1 would be marked stale.

Algorithm 4.3 Memory consolidation and invalidation

```
Ask the model to combine related active assertions into inferences
Require >= 2 distinct supports; check support IDs and subject
Recheck that the supports are still active, then save with links
WHEN a support changes: mark dependent inferences stale (recursively)
```

4.2.4 Hybrid Retrieval and Graph Traversal

This returns separate ranked lists, and the graph expansion stops after two rounds so it doesn't pull in the whole graph (Algorithm 4.4). For E5, keyword and vector search find R2 and the work assertions, and the `supersedes` link of R2 reaches R1 (Chennai).

Algorithm 4.4 Hybrid retrieval and graph traversal

```
Resolve date phrases; run keyword and vector search
Add time-based candidates when there is a date window
Expand from matching assertions and entities for two rounds
Filter by validity and distance; RETURN the candidate lists
```

4.2.5 Reciprocal Rank Fusion and Context Packing

Scores from different searches can't be compared, so only ranks are used (Algorithm 4.5). If R2 is first and R1 second in all three lists, R2 scores $3/61 \approx 0.0492$ and R1 $3/62 \approx 0.0484$, and both fit easily in Pi's 2,048 tokens.

Algorithm 4.5 Reciprocal Rank Fusion and context packing

```
Score(memory) = SUM over lists of 1 / (60 + rank)
Sort by score; break ties by memory ID
Add each memory with its source quotes if it fits the budget
Stop at top_k memories; RETURN the packed context
```

4.2.6 Source-Grounded Answer Generation

Bad output is rejected rather than patched, so every citation the user sees points to a memory that was actually retrieved (Algorithm 4.6). For E5 the answer cites R2 for Bengaluru and R1 for Chennai, so it passes.

Algorithm 4.6 Source-grounded answer generation

```
IF nothing was retrieved: RETURN "Insufficient evidence."
Ask the model to answer only from the retrieved memories
Allow only retrieved IDs as citations; check the format
Require citations when the answer claims enough evidence
```

4.3 INTEGRATION WITH THE PI AGENT

Pi's memory client saves every message with a namespace and session. When Maya asks E5, Pi sends a recall request for `user:maya` (`top_k` 10, `max_tokens` 2,048), and the result is added to the prompt only if it is non-empty and from the same namespace.

CHAPTER 5

RESULTS, PERFORMANCE ANALYSIS AND COMPARISON

This chapter covers the training data, the fine-tuned model and a live demo, and then the benchmark we ran to check whether the memory engine actually helps an agent. At the end we compare our numbers with published memory systems and list what still does not work well.

5.1 DATASETS

Each training row is a short conversation plus a task (extract, consolidate or reflect), and the expected output is the memory for it, with every item tied to a source quote. For example, in the Maya episode the user says "I live in Chennai." and later "Bengaluru is now my home city.", and the expected output is the assertion "Maya, home city, Bengaluru" with `correction:true` and that quote.

The data started with only coding conversations and we later widened it to 16 domains (family, food, travel, education, work, software and so on). All of it is synthetic: the teacher model GPT-5.6-sol generated and labelled the conversations, a second model cross-checked the labels, and we cleaned the result. This gave 2,961 rows, split 2,095 / 424 / 442 for training, validation and testing (Figure 5.1). Many rows check that the model stores only what the user said. If the assistant suggests "You could learn violin this winter." and the user replies that they are learning watercolor painting instead, only the watercolor painting should be stored.

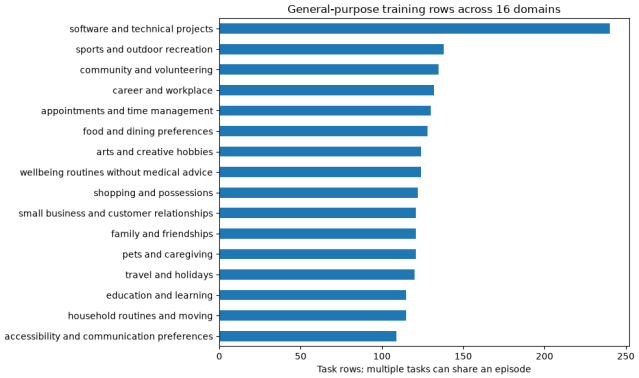


Figure 5.1: General-purpose training rows across the 16 domains

The first benchmark runs (Section 5.5) showed that this data asked for too few facts: most episodes expected only two or three claims, so the model learned to stop early. We therefore made a third set of 480 casual two-person conversations, each labelled with every fact in it by Claude Sonnet 5.5 (5,284 claims). With 600 older rows mixed back in so the other tasks were not forgotten, this gave 1,719 training and 670 validation rows. No LoCoMo text was used anywhere, so the benchmark stays unseen.

5.2 FINE-TUNING AND MODEL SELECTION

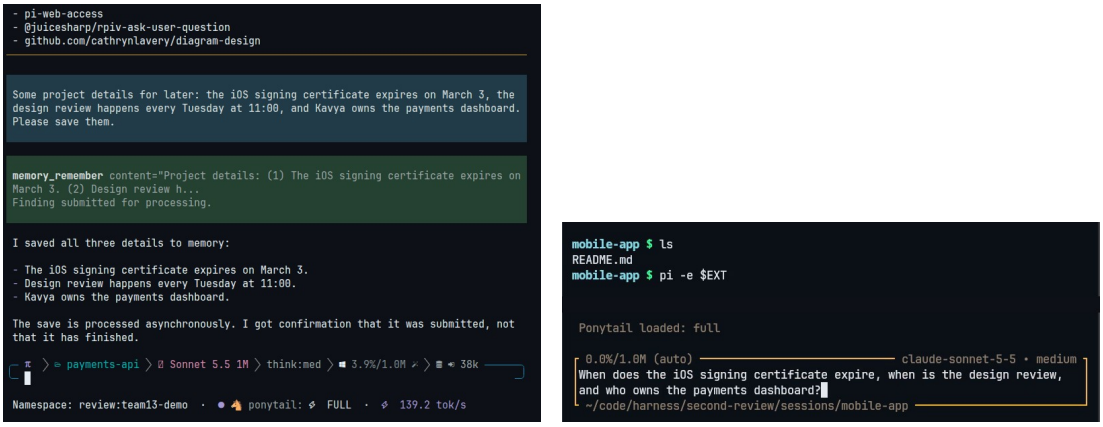
We first tried Qwen3-1.7B. It did fine on synthetic examples but only 18 of 33 real conversation windows passed validation, mostly because of wrong quotes, wrong speakers and invalid IDs. So we moved to Qwen3-4B-Instruct-2507, trained with QLoRA [14] (NF4, rank 16, prompt masked, 4,096-token rows) in two stages, A and B. The adapter was merged, converted to a Q8_0 GGUF with llama.cpp, checked live and imported into Ollama as `mem-extractor:general-v1`.

Stage C continued from Stage B on the coverage set for one epoch, with the row limit raised to 6,144 tokens because the new rows are longer. It

took 2.9 hours on our laptop's 8 GB RTX 4070 (peak 7.3 GB, validation loss 0.155). We saved a checkpoint every 20 steps because an earlier attempt ran out of GPU memory near the end. Before comparing the two models we fixed a simple rule: use the new model, `general-v2`, if it scores at least as well as v1 on the two development conversations. It did, so v2 was used for the test.

5.3 LIVE DEMONSTRATION

The demo works like the Maya example but with its own values: a certificate expiry date, a weekly review time, a dashboard owner and, later, a release-freeze time that gets changed (Figures 5.2 and 5.3).



(a) Recording in the first session (b) Asking in a new session

Figure 5.2: (a) The user gives three details, which are sent for memory processing. (b) A new session in another project asks for them without repeating them



(a) Recalled answer (b) Corrected assertion in the graph

Figure 5.3: (a) The agent answers March 3, Tuesday at 11:00 and Kavya from memory. (b) The new release-freeze time (Friday 16:30) is linked to the old one (Thursday 17:00)

In the new session the agent answers from the recalled assertions and says the information came from an earlier session. It also points out that the stored date has the year 2026 even though the user never gave a year, which is exactly why we show source quotes with every answer. In the graph, the Friday 16:30 freeze is linked to the old Thursday 17:00 value by `supersedes`, the same way Bengaluru replaces Chennai.

5.4 BENCHMARK SETUP

We used LoCoMo [15], the benchmark that Mem0, Zep and A-Mem report on [1]. It has ten long conversations between two people (272 sessions, about 16,000 tokens each) and 1,540 questions with gold answers in four categories: single-hop, multi-hop, temporal and open-domain. Like Mem0 we left out the adversarial category, which has no gold answers. Each conversation is fed into the engine session by session as its own memory bank. Then every question is answered by the same reader model with the same prompt, and only

the context it is given changes:

- **Memory engine:** the engine's recall (assertions with quotes, inferences, graph neighbours and source text), up to 8,192 tokens.
- **Plain retrieval:** keyword and vector search over the raw text only, same budget. This is normal RAG.
- **Recent window:** the last 8,192 tokens of the conversation, i.e. what an agent keeps when its history overflows.
- **Whole conversation:** everything (the upper reference).
- **No memory:** just the question.
- **Ablations:** the engine without source text, without the graph, or without consolidation, on a 300-question sample (75 per category).

The reader is Claude Haiku 5.5, run through the Pi agent, with Mem0's answer prompt. Each answer is graded against the gold answer by two judges using Mem0's LoCoMo rubric, and neither judge knows which condition the answer came from. Our main judge is TypeSafe Jev [20], which gives the probability that an answer is correct (we count ≥ 0.5 as correct). The second is GPT-4o-mini, called the same way as in Mem0's code so we can compare with their table. For differences between conditions we use paired bootstrap 95% intervals [21], and for judge agreement Cohen's kappa [22].

To keep ourselves honest we wrote the conditions and pass marks into the repository before the test run, and the run script refuses to continue if the code changes in the middle. We only debugged on two of the ten conversations (the dev split). The whole run is one command (`benchmark/overnight.sh`) and took 7 h 45 min on the laptop. Every model call is saved, a `verify` command checks that no stored answer was changed, and a Jupyter notebook

(`benchmark/Benchmark_Report.ipynb`) recomputes all the numbers below from the saved records.

5.5 DEVELOPMENT ITERATIONS

The first runs on the two dev conversations found several bugs, which we fixed and logged with before and after numbers in `benchmark/iterations.json`. A busy session made the extractor's output too long, so such windows are now split and retried. "Yesterday" was stored as the day it was said. Consolidation prompts stopped fitting the model after about a dozen sessions (18 of 28 jobs failed). The review step threw away real corrections such as "I moved from Hobart to Accra". Most importantly, recall made only of assertions scored 45.9% against 79.8% for plain retrieval, because the extractor missed many facts. Mixing source text into recall brought it to 82.8% against 82.4%, and the Stage C training above raised assertions alone from 51.9% to 63.1%. We also fixed three bugs in our own benchmark script. The dev numbers are on only 233 questions, so a change of a point or two there is just noise.

5.6 RESULTS

Table 5.1 shows the main result. With the engine the reader got 83.1% of the questions right. With no memory it got almost nothing (0.3%), since the questions are about details only the conversation contains. Keeping just the last 8,192 tokens gave 43.0%, because most answers are in older sessions. Plain retrieval gave 80.0%, and the whole conversation (about 22,400 tokens) gave 89.0%. So the engine closes most of the gap between plain retrieval and the whole conversation while sending only 30% of its tokens. All the pass marks we had fixed in advance were met.

Table 5.1: LoCoMo results by condition (all ten conversations, categories 1–4)

Condition	n	Jev	GPT-4o-mini	F1	BLEU-1	Tokens	Reader s
Memory engine	1540	83.1	83.2	55.8	48.4	6,790	5.2
Plain retrieval	1540	80.0	80.1	53.5	46.4	4,473	5.5
Recent window	1540	43.0	42.3	28.9	25.0	8,168	5.2
Whole conversation	1540	89.0	88.8	60.4	52.6	22,400	5.5
No memory	1540	0.3	0.4	0.5	2.0	7	4.1
Assertions only	300	66.7	66.7	41.3	35.7	5,566	5.2
Without graph	300	76.3	76.7	47.9	41.6	6,606	5.3
Without consolidation	300	74.7	75.7	47.3	40.5	6,635	5.1

Jev and GPT-4o-mini: percentage of answers judged correct against the gold answer with Mem0's LoCoMo rubric. F1 and BLEU-1: Mem0's token overlap with the gold answer. Context tokens: mean tokens given to the reader besides the question. Reader s: median seconds per answer. The last three rows are on the 300-question stratified sample.

The engine is 3.1 points ahead of plain retrieval, with a 95% interval of [1.2, 4.8], so the gain is small but real. It is 6.0 points behind the whole conversation. GPT-4o-mini gives almost the same numbers: the two judges agree on 96.7% of the answers (kappa 0.93). Since we debugged on conversations 26 and 30, we also checked the other eight alone: 83.0% for the engine, 79.4% for plain retrieval and 88.9% for the whole conversation, so the result is not just from tuning to the dev conversations. Asking the same 300 questions three times, the engine's accuracy changed by only 0.5 points (standard deviation) and plain retrieval's by 1.3.

Table 5.2: Accuracy by LoCoMo question category (Jev, %)

Category	n	Memory engine	Plain retrieval	Recent window	Whole conversation	No memory
Single-hop	841	87.0	83.9	44.9	93.3	0.0
Multi-hop	282	83.0	74.5	51.4	88.7	0.4
Temporal	321	80.1	80.4	31.2	85.7	0.0
Open-domain	96	58.3	60.4	40.6	63.5	3.1

Table 5.2 shows that most of the gain comes from multi-hop questions (83.0% against 74.5%), where the answer is spread over several sessions. For example, for “What subject have Caroline and Melanie both painted?”, plain retrieval led the reader to answer “Nature”. The engine’s recall had one short assertion per person (“Caroline painted a sunset scene …” and “Melanie painted a sunset with a palm tree …”), and the reader answered “Sunsets”. An assertion sums up in one line what took several turns, so more facts fit in the budget. On temporal questions the two are level (80.1% against 80.4%), even though this is where we expected the engine to help most. Section 5.8 explains part of the reason.

The ablations (last three rows of Table 5.1) show that assertions alone are not enough yet: removing the source text costs 9.7 points, because the extractor still misses facts. Removing the graph or consolidation made no difference we could measure. LoCoMo’s questions are about two people and rarely need a chain of relations, so this benchmark cannot really show what the graph is for.

5.7 PERFORMANCE ANALYSIS

Retrieval. Scored against LoCoMo’s evidence labels, the engine’s context contained the turn that answers the question for 84.4% of questions, against 80.6% for plain retrieval. This is because an assertion carries the quote of the turn it came from. All 99,971 source quotes the engine returned matched the original text exactly, so every answer can be traced back to what was actually said.

Tokens and speed. The engine sends 6,790 tokens of context per question against 22,400 for the whole conversation. Counting the prompt and answer, a question costs 14,900 tokens instead of 35,300. LoCoMo’s

conversations still fit in Haiku's context window, so the reader takes about the same time either way (median 5.2 s with the engine, 5.5 s with the whole conversation). The real advantage is that the engine's context stays under 8,192 tokens however long the history gets, while the whole conversation keeps growing. Recall itself takes 428 ms at the median and 584 ms at p95.

Building memory. This is the slow part. The 272 sessions (5,882 turns) took 4.2 hours on the laptop, about 56 seconds per session, almost all of it extraction by the 4B model. No job failed for good. It happens in the background when a session is saved, so it does not slow down the agent's answers.

5.8 PROPERTY TESTS

LoCoMo cannot check things like whether an old value is kept after a correction. So we also wrote 40 scripted conversations that check the engine's stored state directly, with no judge. The engine refused to answer without evidence in all 6 cases, never stored an API key, and after each of the 12 updates it returned the new value while `as_of` recall still returned the old one.

These tests also caught a bug that the LoCoMo judge could not see: none of the 12 relative dates (like "yesterday") were resolved in the test run. The new model writes the date into the sentence but often leaves the date field empty, and the store then filled it with the time the message was sent. We fixed this after the test run (commit `43af5bd`), and on re-running the tests all 10 events that were extracted have the right date. The LoCoMo numbers above are from before this fix. The other failures were all facts the extractor never produced, which is the same coverage problem the ablation shows.

5.9 COMPARISON WITH EXISTING SYSTEMS

Table 5.3 puts our result next to the LoCoMo numbers published in the Mem0 paper [1], which also covers Zep [5], LangMem [23] and A-Mem [24]. The raw scores cannot be compared directly, because they used GPT-4o-mini as the reader and we used Claude Haiku 5.5, which is stronger and raises every condition (our whole-conversation score is 88.8% against their 72.9%). A fairer check is how far each system is from the whole-conversation score in its own study (last column). There we are 5.6 points behind, compared with 4.5 for Mem0 with a graph, 6.0 for Mem0 and 6.9 for Zep. So we are in the same range as Mem0 and Zep, not ahead of them, and we use more context tokens to get there.

Table 5.3: Published LoCoMo results [1] beside this work (GPT-4o-mini judge)

System	J (%)	Context tokens	J minus own full context
Full-context	72.90	26,031	+0.00
Mem0g	68.44	3,616	-4.46
Mem0	66.88	1,764	-6.02
Zep	65.99	3,911	-6.91
RAG (best, k=2, 256-token chunks)	60.97	–	-11.93
LangMem	58.10	127	-14.80
OpenAI	52.90	4,437	-20.00
A-Mem	48.38	2,520	-24.52
This work: whole conversation (Haiku 5.5)	88.83	22,400	+0.00
This work: memory engine (Haiku 5.5)	83.18	6,790	-5.65
This work: plain retrieval (Haiku 5.5)	80.06	4,473	-8.77

Published systems answer with GPT-4o-mini; this work answers with Claude Haiku 5.5, so absolute scores are not comparable. The last column compares each system with the full-context baseline of its own study. Context tokens for published systems are from the Mem0 paper.

Where we do differ is in what is kept. Mem0 and A-Mem overwrite or delete a memory when it is updated, while our engine keeps every version

with its dates and source quote, so it can answer questions about the past and every answer can be checked. It also runs fully on one laptop with a 4B model, while the published systems call a hosted model for every memory operation.

5.10 LIMITATIONS AND FUTURE WORK

We built all five modules and showed on LoCoMo that the engine helps: 83.1% against 80.0% for plain retrieval and 43.0% for a recent window, and 6.0 points below the whole conversation with 30% of the tokens. But there are clear limits:

- We only used one benchmark with ten conversations. LongMemEval was planned but not run, and we did not test on real coding sessions.
- Answers were graded by two model judges, not by people. The notebook prints random items so anyone can check them by hand.
- We used one reader model. A weaker reader may depend on memory more, a stronger one less.
- Plain retrieval used fewer tokens than the engine (4,473 against 6,790), so giving it more chunks might close some of the 3.1-point gap.
- The extractor still misses facts (assertions alone get 66.7%), and the graph and consolidation made no measurable difference on LoCoMo.

For future work we want to improve extraction coverage and speed (M2), add access control and backups (M1), test updates and aliases on a benchmark made for them (M3), run LongMemEval and a coding benchmark (M4), and build a proper date test set with human checking of the judges (M5).

REFERENCES

[1] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, Oct. 2025, pp. 2993–3000, also arXiv:2504.19413 [cs.CL].

[2] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[3] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 86 809–86 836, also arXiv:2410.10813. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, Oct. 2023, (rev. Feb. 2024). [Online]. Available: https://arxiv.org/abs/2310.08560

[5] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv preprint arXiv:2501.13956 [cs.CL], Jan. 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[6] S. Daga, S. Sreedhar, and D. Shah, "LongMemEval," supermemory research (vendor-hosted evaluation report on LongMemEval-S, not peer reviewed), May 2026, (accessed Oct. 6, 2026). [Online]. Available: https://supermemory.ai/research/longmembench/

[7] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, "Hindsight is 20/20: Building agent memory that retains, recalls, and reflects," arXiv preprint arXiv:2512.12818 [cs.CL], Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.12818

[8] Neo4j, Inc., "Query the database," in *Neo4j HTTP API Manual, Version 5* (page marked deprecated in 5.26). Neo4j, Inc., 2026, (accessed Oct. 6, 2026). [Online]. Available: https://neo4j.com/docs/http-api/5/query/

[9] M. Zechner *et al.*, "pi: AI agent toolkit (unified LLM API, agent loop, TUI, coding agent CLI)," package @earendil-works/pi-coding-agent, v1.0.2 (version locked by the project's package-lock.json; latest is 1.0.4 on Oct. 6, 2026), GitHub repository, 2026, (accessed Oct. 6, 2026). [Online]. Available: https://github.com/earendil-works/pi

[10] PostgreSQL Global Development Group, "PostgreSQL 17 documentation, chapter 12: Full text search (sections 12.2.2, 12.3.2, 12.3.3, 12.9) and section 64.4: GIN indexes," PostgreSQL 17.11 documentation, 2026, (accessed Oct. 6, 2026). [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html

[11] A. Kane *et al.*, "pgvector: Open-source vector similarity search for Postgres," source code repository and README (0.8.6 installed in the test database, v0.8.7 tag also released), Oct. 2026, (accessed Oct. 6, 2026). [Online]. Available: https://github.com/pgvector/pgvector

[12] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09)*, Boston, MA, USA, Jul. 2009, pp. 758–759.

[13] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv preprint arXiv:2506.05176 [cs.CL], Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 10 088–10 115, also arXiv:2305.14314.

[15] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Aug. 2024, pp. 13 851–13 870, (preprint: arXiv:2402.17753). [Online]. Available: https://aclanthology.org/2024.acl-long.747/

[16] Wikimedia Foundation, "Wikibase/DataModel," MediaWiki.org, 2026, (accessed Oct. 20, 2026). [Online]. Available: https://www.mediawiki.org/wiki/Wikibase/DataModel

[17] T. Lebo, S. Sahoo, and D. McGuinness, "PROV-O: The PROV ontology," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-o/

[18] T. Mitchell, W. Cohen, E. Hruschka, P. Talukdar, B. Yang, J. Betteridge, A. Carlson, B. Dalvi, M. Gardner, B. Kisiel, J. Krishnamurthy, N. Lao, K. Mazaitis, T. Mohamed, N. Nakashole, E. Platanios, A. Ritter, M. Samadi, B. Settles, R. Wang, D. Wijaya, A. Gupta, X. Chen, A. Saparov, M. Greaves, and J. Welling, "Never-ending learning," *Commun. ACM*, vol. 61, no. 5, pp. 103–115, May 2018.

[19] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST '23)*, 2023, also arXiv:2304.03442.

[20] TypeSafe AI, "SystemOne API: the Jev decision model," https://api.typesafe.ai/v1/systemone, 2026, model alias jev-latest. Accessed: Oct. 21, 2026.

[21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[22] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.

[23] LangChain, "LangMem SDK: Long-term memory for agents," https://github.com/langchain-ai/langmem, 2025, accessed: Oct. 20, 2026.

[24] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," in *Proc. 39th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025, arXiv:2502.12110 [cs.CL].